Module

1

Introduction to Computer Organization and Architecture

Introduces the fundamental structure of a computer system, including architectural models, functional units, number representation, registers, buses, and instruction set design concepts such as RISC and CISC.

1.1

Computer Organization and Architecture | Course Orientation

Explore how computer organization and architecture connect software execution with hardware behavior.

1.2

Optional

Computer Evolution: Historical Development and Generations of Computers

Investigate how advances in hardware technology shaped the evolution of modern computing systems.

1.3

Von Neumann and Harvard System Models

Compare memory organization and instruction flow in Von Neumann and Harvard architectures.

1.4

Computer Functional Units: CPU, Memory, I/O, and Bus Structures

Examine how CPU, memory, I/O, and buses coordinate during program execution.

1.5

Optional

Assignment | Computer Functional Units

This is an assignment to test the understanding of the previous lesson.

1.6

Number Systems

Interpret and convert data across binary, octal, decimal, and hexadecimal number systems.

1.7

Optional

Binary Codes

Analyze how binary coding schemes are used to represent characters, digits, and error-checking information.

1.8

Signed Number Representation

Analyze how computers represent positive and negative numbers using complement systems.

1.9

Optional

Logic Gates and Boolean Algebra

Investigate how logic gates and Boolean operations form the foundation of digital circuits.

1.10

Optional

Assignment | Logic Gates and Boolean Algebra

This is an assignment to test the understanding of the previous lesson.

1.11

Registers and Common Bus Organization

Trace how data moves internally between registers using common bus organization.

1.12

Optional

Assignment | Registers and Common Bus Organization

This is an assignment to test the understanding of the previous lesson.

1.13

Optional

Assignment | Registers and Common Bus Organization - Implementation

This is an assignment to test the understanding of the previous lesson.

1.14

Instruction Set Architecture and RISC vs CISC Design

Investigate how instruction sets define communication between software and processor hardware.

1.15

Flynn’s Classification of Parallel Computer Systems

Classify parallel computing systems using Flynn’s taxonomy.

1.16

Optional

Assignment | Flynns Classification of Parallel Computer Systems

This is an assignment to test the understanding of the previous lesson.

1.17

Optional

Assignment | Processor Architectures and Parallel Computing

Apply concepts of ISA, RISC vs CISC design, and Flynn’s Classification to analyze modern computing systems.

1.18

Optional

Introduction to Embedded Systems and Microcontrollers

Explore how embedded systems and microcontrollers are designed to perform specialized computing tasks in real-world devices.

Module

2

Central Processing Unit Design and Instruction Execution

Explores how instructions are executed within the CPU, covering the instruction cycle, addressing modes, instruction formats, datapath design, and control unit implementation.

2.1

The Instruction Cycle

Examine how instructions move through fetch, decode, execute, and write-back stages inside the CPU.

2.2

Instruction Formats and Instruction Types

Investigate how instruction formats and instruction categories define processor operations.

2.3

Addressing Modes

Analyze how different addressing modes locate operands during instruction execution.

2.4

Optional

Assignment | Addressing Modes

This is an assignment to test the understanding of the previous lesson.

2.5

Optional

Instruction Decoder and Addressing Mode Implementation

This is an assignment to test the understanding of the previous lesson.

2.6

Optional

Branch Instructions and Program Control Flow

Explore how branch instructions alter program execution flow and modify processor control behavior.

2.7

Optional

Assembly Language Programming and Instruction Sequencing

Interpret simple assembly programs and analyze instruction sequencing behavior.

2.8

Hands-on: Tracing Instruction Execution with CPU Simulator

Apply instruction execution concepts using a CPU simulator environment.

2.9

Optional

Hands-on: Tracing Instruction Execution with CPU Simulator - Problem 2

Trace instruction execution using a CPU simulator and analyze changes in register and memory states.

2.10

Data Path Design and Execution Flow

Examine how datapath components coordinate instruction execution inside the processor.

2.11

Optional

Assignment | Data Path Design and Execution Flow

This is an assignment to test the understanding of the previous lesson.

2.12

Optional

Microoperations and Register Transfer Language (RTL)

Investigate how microoperations and RTL describe internal processor data movement.

2.13

Control Unit Design: Hardwired Control

Examine how hardwired control units generate processor control signals.

2.14

Control Unit Design: Microprogrammed Control

Analyze how microprogrammed control units execute instructions using control memory.

2.15

Optional

Assignment | Control Unit Design

This is an assignment to test the understanding of the previous lesson.

2.16

Optional

Comparing Control Unit Designs: Trade-offs and Real-World Practice

Evaluate trade-offs between hardwired and microprogrammed control implementations.

2.17

Optional

Interrupts and Interrupt Handling Mechanism

Investigate how processors respond to external and internal interrupt signals.

2.18

Case Study: x86, MIPS, and RISC-V Instruction Set Architectures

Relate CPU architecture concepts to real-world processor instruction set designs.

Module

3

Pipelining and CPU Performance

Examines how pipelining improves processor performance, identifies different types of hazards, and applies performance evaluation techniques such as CPI and Amdahl's Law.

3.1

Pipelining: Stages, Throughput, and Speedup

Examine how pipelining improves instruction execution efficiency and processor throughput.

3.2

Optional

Assignment | Pipelining: Concept, Stages, Throughput, and Speedup

This is an assignment to test the understanding of the previous lesson.

3.3

Pipeline Performance Parameters

Analyze how pipeline depth, clock cycles, and CPI influence processor performance.

3.4

Optional

Asynchronous Pipeline Design

Explore advanced pipeline organizations that operate without synchronized clock stages.

3.5

Structural Hazards

Investigate hardware resource conflicts that occur during pipelined execution.

3.6

Optional

Assignment | Structural Hazards

This is an assignment to test the understanding of the previous lesson.

3.7

Data Hazards

Analyze instruction dependency problems that occur in pipelined processors.

3.8

Optional

Advanced Hazard Resolution

Examine advanced techniques used to minimize pipeline stalls and execution delays.

3.9

Control Hazards

Examine how branch instructions affect pipeline execution flow and processor performance.

3.10

Optional

Assignment | Control Hazards

This is an assignment to test the understanding of the previous lesson.

3.11

Optional

Branch Target Buffers and Speculative Execution

Investigate advanced branch handling techniques used in modern processors.

3.12

Pipeline Performance Analysis with Hazards

Evaluate the impact of hazards on pipeline efficiency and processor performance.

3.13

Optional

Superscalar Processors and Out-of-Order Execution

Explore advanced processor execution techniques that improve instruction-level parallelism.

3.14

CPU Performance Equation

Relate instruction count, CPI, and clock rate to overall CPU performance.

3.15

Amdahl's Law and Optimization Limits

Analyze the limits of performance improvement in optimized computing systems.

3.16

Optional

Case Study: Pipelining Optimization in Modern Processors

Relate pipelining concepts and hazard mitigation techniques to modern processor architectures.